1. Research Question

1.1 Is Stemming beneficial in improving performance?

Stemming is the process of extracting the root word from a given inflection word. It also has a significant role in a variety of Natural Language Processing applications (NPL). Stemming algorithms or stemmers are the terms used to describe stemming programs.
A stemming algorithm reduces the words “chocolates,” “chocolatey,” “choco” to the root word, “chocolate, and “retrieval,” “retrieved,” “retrieves” reduce to the stem “retrieve.”
Stemming is the method of eliminating affixes from words to recover the base form through stemming from building a robust model. Stemming minimizes the index size and improves retrieval.
There are two types of errors in stemming, Over-stemming, and Under-stemming. Stemming is widely used in search engines, text mining, and in information retrieval systems.

2. Data Mining Process

Once we open the Weka software, you will see a dialogue box with four buttons.
Click on “Explorer” to start working with the dataset. The first step is to load the dataset using Weka, as the data set is in text format, a pop-up comes up which shows “Cannot determine file loader automatically. Please choose one.” Click Ok which will navigate to **Weka GUI (Graphic User Interface) (GenericObjectEditor).** Now click on choose and set Weka/ Core/ Converters/ “**TextDirectoryLoader**.” Now save the file in. arff format.
We can now see that the dialogue box consists of 2 categories ‘text’ and @@class@@. There are two classes one is **business** class which consists of around 510 text files and the other class **tech** has 401 text files. We can clearly observe that the dataset is imbalanced, and it should be balanced for better performance.

2.1 Data Balancing

We can observe a histogram that shows the data is imbalanced i.e., 510 for the business class and 401 for the tech class. The business class has more text files than the tech class. The data can be balanced by performing the **SMOTE** instance.
We need to click on Choose **Filter / supervised / instance / SMOTE**
SMOTE is a tool that can be used in Weka to increase the minority group when data imbalance occurs. The SMOTE filter is used to increase classifier performance despite imbalanced datasets.
We need to change the parameters of SMOTE filter
-> classValue – 0 (minority class)
-> nearest neighbors – 5 (produced instanced by calculating 5 neighbors by placing them within those 5 neighbors).
-> percentage – 33 (instances in minority class will be increased by 33%)
-> randomSeed – 1
-> Now click Ok and save the file in. arff format
We can now observe that the data is balanced i.e., the business class has 510 text files, and the tech class has 531 text files.

2.2 Conversion without Stemming

Now we need to perform the conversion using the **'stringtoWordVector'** filter and after applying the filter changes consider this as a dataset (1).
We need to change parameters in the **"stringtoWordVector"** as shown below.
-> IDF Transform – True (way to find words and documents that are strongly related)
-> TF Transform – True
-> minTermFreq – 1 (filters the attributes and removes them if the word repeats less than the given value
-> OutputWordCounts – True (display frequency of each word)
-> Stemmer – NullStemmer (Stemmer setting, which indicates that no stemmer algorithm has been used)
-> Stopwords – Weka
-> UseStopList – True (filter all words in the Weka directory)
Click ok and apply the filter.
There are 1043 instances and 1648 attributes after applying the filter with the above parameters and removing unnecessary symbols.

2.3 Conversion with stemming

Now using the same parameters as above, the field stemmer needs to be changed to **'LovinsStemmer.'** The stemmer performs better and is suitable for most documents.
We need to set the parameters as shown below.
-> IDF Tranform - True
-> TF Tranform – True
-> minTermFreg - 1
-> OutputWordCounts – True
-> Stemmer – LovinsStemmer
-> Stopwords – weka
-> UseStoplist – True
Lovins stemmer is used as a stemming algorithm, but the remaining properties remain the same.
After applying the stemming filter, we can observe a slight reduction in the attributes.

3. Testing and Training

We use the **Decision tree, Naïve Bayes, and Support vector machine (SVM)** to perform the testing and training sets. To improve the accuracy of the data sets we need to take at least two accuracy values by changing the seed value. We need to set seed value 01 as accuracy1 and seed value 20 as accuracy. Finally, implement the average of accuracies to get the final accuracy.

3.1 Implementation of Decision Tree in Weka

-> Click on the “Classify” tab on the top navigation.
-> Now select the “choose” button.
-> From the drop-down list. Select “trees” which contains all the tree algorithms.
-> Finally, we need to select the “J48” which is the Decision Tree.
-> Set percentage split from 10% - 90% under the test options.
-> Set the seed values to 01 and 20 from the “more options”
-> Applying the changes, we get the different accuracy results based on the different split values.

3.2 Implementation of Naïve bayes in Weka

-> Click on the “Classify” tab on the top navigation.
-> Now select the “choose” button.
-> Select “Bayes” from the drop-down menu.
-> Finally select “NaiveBayes.”
-> Set percentage split from 10% - 90% under the test options.
-> Set the seed values to 01 and 20 from the “more options”
-> Applying the changes, we get the different accuracy results based on the different split values.

3.3 Implementation of Support Vector Machine in Weka

-> To implement the Support vector machine, we need to use the “GridSearch” to get good values and set the parameters as shown below.
-> Click on the “Classify” tab on the top navigation.
-> Now click on the choose button.
-> From the drop-down list select “meta.”
-> Select “GridSearch” a process to find correct and optimal parameters. Set the parameters as shown below.
-> Xbase and Ybase – 10
-> Xexpression and Yexpression – pow (BASE, I)
-> Xmax and Ymax – 5.0
-> Xmin and Ymin - -2.0
-> Xproperty – classifier.cost
-> Yproperty – classifier.gamma
-> Set the classifier -> LibSVM
-> Kernal – radial basis
-> normalize – True
-> Evalutaion – Accuracy
-> Filter – ALLfilter

3.4 Analysis of non-stemmed data

Split	Decision Tree			Naive Bayes			SVM		
	Accuracy1 Seed-01	Accracy2 Seed-20	Average	Accuracy1 Seed-01	Accuracy2 Seed-20	Average	Accuracy1 Seed-01	Accuracy2 Seed-20	Average
10	80.9	78.7	79.58	95.4	95.1	95.2	95.4	95	95.2
20	87	83.2	85.1	95.6	96.5	96	97.5	97.7	97.6
30	88.5	91.1	89.8	94	97	95.5	98	97.9	97.9
40	90.5	92	91.2	95	97.1	96	97	98.6	97.8
50	90.9	91	90.9	95.1	96.7	95.9	98	99	98.5
60	91.5	93	92.2	96.6	96.8	96.7	99	98	98.5
70	91	92	91.5	96.2	97.2	96.7	98.2	98.2	98.2
80	93	93.1	93	96.7	97.4	97.2	96.5	99	97.7
90	91	91.3	91.1	98	97	97.5	97	99	98

Table 3.4.1: Accuracy percentage of the classifier with non-stemmed data

split	Decision Tree	Naïve bayes	SVM
10	79.58	95.2	95.2
20	85.1	96	97.6
30	89.8	95.5	97.9
40	91.2	96	97.8
50	90.9	95.9	98.5
60	92.2	96.7	98.5
70	91.5	96.7	98.2
80	93	97.2	97.7
90	91.1	97.5	98

Table 3.4.2: Overall Averages of classifiers with non-stemmed data

From the above table, we can observe that the **Decision tree, Naïve Bayes, and Support vector machine (SVM)** the performance of all the algorithms with differences in the classifiers.

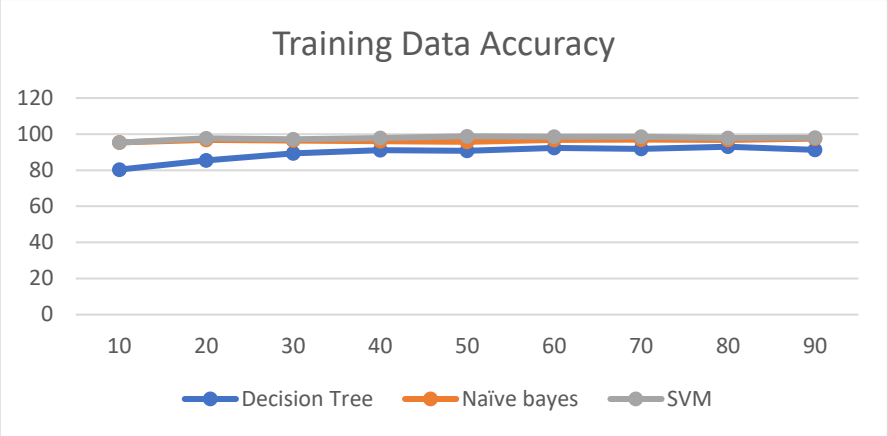


Figure 3.4.3: Graph plotting using an average of Non – Stemmed Data

The above graph has been plotted based on the overall averages of all the classifiers using Non stemmed data. From Figure 3.4.3 we can observe that J48, SVM, and Naïve Bayes have performed well with minute differences. From the graph plotted above, we can say Naïve Bayes and SVM have the same performance, Decision tree has performed a little less compared to the other two classifiers.

3.5 Analysis of Stemmed data

Split	Decision Tree			Naive Bayes			SVM		
	Accuracy1 Seed-01	Accracy2 Seed-20	Average	Accuracy1 Seed-01	Accuracy2 Seed-20	Average	Accuracy1 Seed-01	Accuracy2 Seed-20	Average
10	80.9	79.8	80.3	95.7	95.1	95.4	95.1	95.5	95.3
20	87.2	83.8	85.5	96.9	96.7	96.8	97.7	97.5	97.6
30	88.8	89.9	89.3	95.8	97.2	96.5	98.8	97.7	97.2
40	90.3	91.9	91.1	95	97.1	96	97.2	98.5	97.8
50	90.9	90.7	90.8	95.9	95.7	95.8	98.6	99	98.8
60	91.8	93.0	92.4	96.6	96.8	96.7	99	98	98.5
70	91.6	92.3	91.9	96.4	97.4	96.9	98.7	98.3	98.5
80	93.2	93.2	93	96.1	97.5	96.8	96.6	99	97.8
90	91.3	91.3	91.3	98	97.1	97.5	97.1	99	98

Table 3.5.1: Accuracy percentage of all classifiers with Stemmed data

split	Decision Tree	Naïve bayes	SVM
10	80.35	95.4	95.3
20	85.5	96.8	97.6
30	89.3	96.5	97.2
40	91.1	96	97.8
50	90.8	95.8	98.8
60	92.4	96.7	98.5
70	91.9	96.9	98.5
80	93	96.8	97.8
90	91.3	97.5	98

Table 3.5.2: Overall averages of all classifiers with stemmed data.

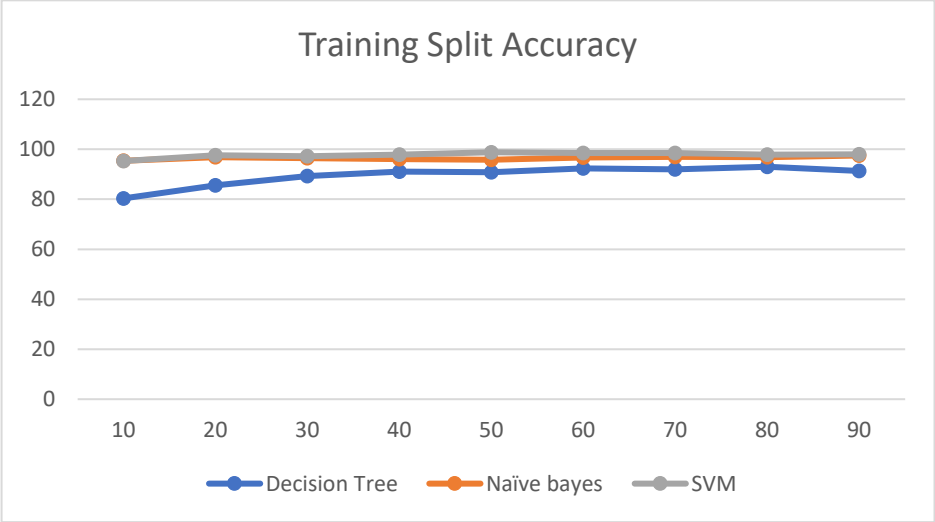
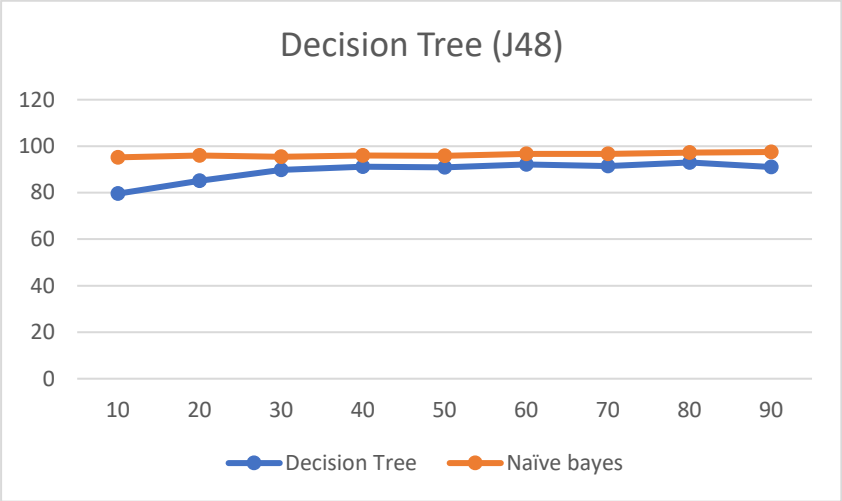


Table 3.5.3: Accuracy with training set using stemmed data.

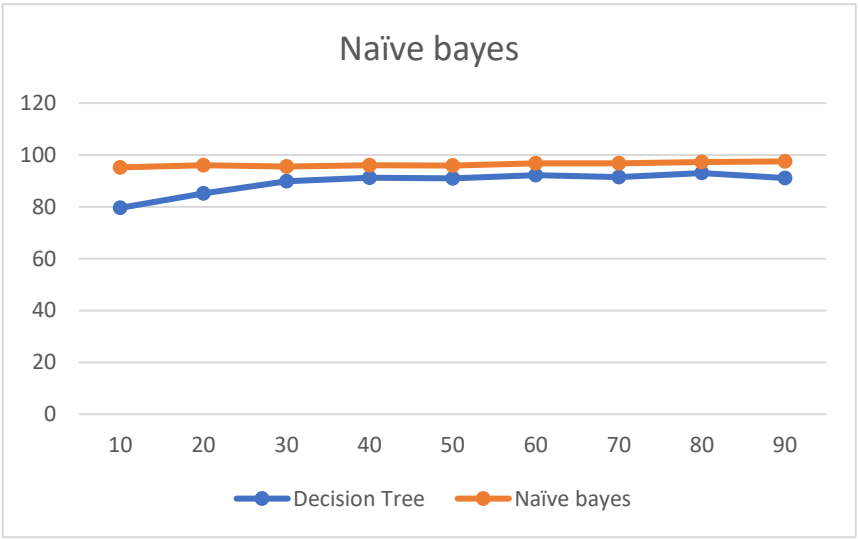
The above graph has been plotted based on the overall averages of all the classifiers with stemmed data. We can observe that SVM and Naïve Bayes have almost performed the same while Decision tree performance was less compared with the other classifiers.

4. Conclusion

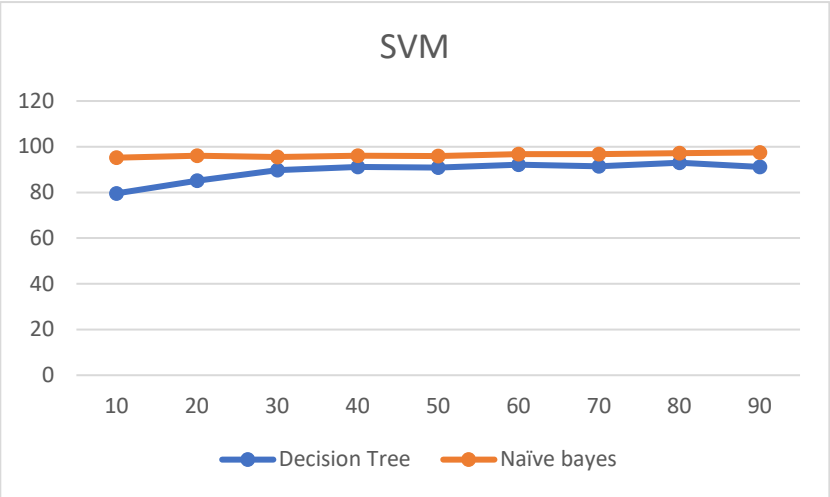
From the below graph we observe that Decision Tree (J48) has the same performance using stemmed and non-stemmed data. Only at a few splits, did we observe the low performance we observed for both the data sets at 1 and have the same performance.



Now for Naïve Bayes, the graph below shows the accuracy of Naïve Bayes' non–stemmed data and stemmed data. Both the datasets have achieved the same accuracy. We observe that at split 10 both the stemmed and non-stemmed data have the same performance. There is a slight difference in the values in all the independent splits.



Finally, for SVM, from the below graph we observe that the accuracy values for the non-stemmed dataset have performed well compared to stemmed data. At some point, we can say that there are minute differences in the performance using the stemmed and non-stemmed data. For SVM, the stemmed data might not be beneficial for this algorithm.



Based on the above analysis, comparing all the performance of different classifiers. We can conclude that Naïve Bayes, Decision trees (J48), and SVM classifiers have performed a bit better in stemmed data compared to non-stemmed data. Through this investigation, we can clearly say that the Decision tree (J48) algorithm is beneficial to consider for performing a better model.